

137. Le monde glisse, le héros ne bouge pas

La position dans le monde vaut « défilement + 76 ».

1. Ce qu'il faut comprendre

Le lutin ne bouge **jamais** : il est à 76 pixels, image après image. Ce qui change, c'est `defiler()`.

C'est ce qui distingue un écran fixe d'un monde. La position du personnage **dans le monde** vaut `monde + 76`, et c'est ce nombre-là qu'on compare au décor — pas celui du lutin, qui est une constante.

La leçon « La caméra : c'est le monde qui bouge » part du même principe et le pousse jusqu'au bord du niveau.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **397 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	119 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	40 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	32 o	← CopierTuiles — 1 lettre sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

sprite()	× 1	23 o sur place, dans main()
poser()	× 1	13 o sur place, dans main() + AdresseCase (10 o)
bouton()	× 2	10 o sur place, dans main() + LireManette (37 o)
defiler()	× 1	7 o sur place, dans main()
image()	× 1	6 o sur place, dans main() + AttendreImage (28 o)

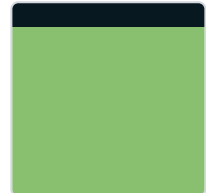
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : GAUCHE / DROITE font glisser le sol ; le héros, lui, reste
// au milieu. On a l'impression qu'il marche.
//
// defiler(x, 0) décale tout le fond de x pixels. Les lutins ne défilent pas :
// le héros reste à sa place à l'écran pendant que le monde passe.

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <poser>      // pose une tuile sur une case du fond
#include <bouton>     // lit un bouton de la manette
#include <defiler>    // fait glisser tout le fond
#include <sprite>     // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — SOL

```
Tuile SOL = {
    // Un trait sombre en haut, puis du moyen clair.
    "#####", "-----", "-----", "-----",
    "-----", "-----", "-----", "-----",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le jeu commence ici.
    for (uint8_t x = 0; x < 20; x++) // chaque colonne...
        for (uint8_t y = 14; y < 18; y++) // ... lignes 14 à 17 :
            poser(x, y, SOL);           // du sol.

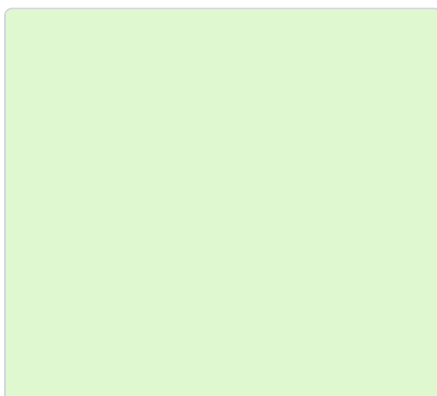
    uint8_t monde = 0;                // De combien le monde a glissé (en pixels).

    while (true) {                    // Répète pour toujours :
        if (bouton(DROITE)) monde++;  // DROITE : le monde glisse (le héros « avance »)
        if (bouton(GAUCHE)) monde--;  // GAUCHE : dans l'autre sens

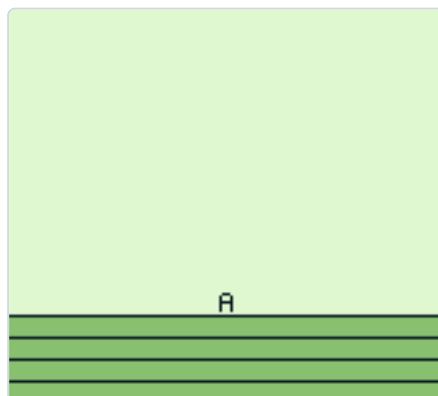
        defiler(monde, 0);             // On décale le fond.
        sprite(0, 76, 104, 1);        // Le héros (la tuile 1), toujours au même endroit.
        image();                       // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



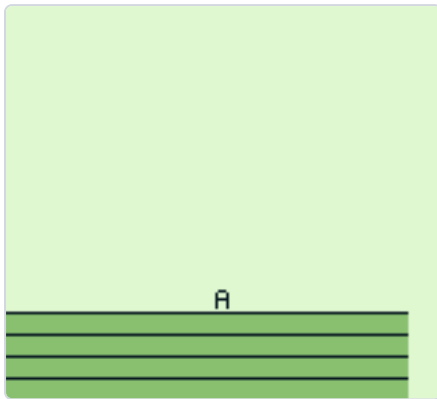
la toute première image — la console vient de démarrer — image 1



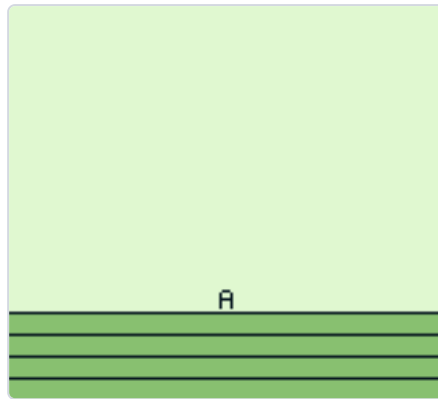
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Le décor glisse ; le personnage reste rigoureusement au milieu.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le monde a glissé ($0 \rightarrow 40$)
- ✓ le lutin n'a pas bougé d'un pixel ($x = 76$)
- ✓ il est resté au milieu
- ✓ le sol est posé

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

SOL

+ tuile 8 × 8

+ perso 16 × 16

+ perso 32 × 32

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

SOL

tuile n° 44 — poser(x, y, SO

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

×2

÷2

Agrandir...

Vider le dessin

Taille voulue :

✓ Créer la copie

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

SOL ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

SOL peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre...

Ensuite : **138. Un record qui survit à l'extinction** — chapitre 24 : Aller au bout. · gameboy3 — écrire une cartouche Game Boy en C++